Scrimmagine

Design Document

Jai Barath
Nick Van Wagner

Purpose

Scrimmagine drops players into the biggest moments of real NFL games and lets them make the call. Every result is simulated from real historical plays, and the game scores you against the coach who actually made the decision.

Fans second-guess play calls every Sunday, but they never find out if their call would have worked. Full football sims make you play 60 minutes to reach the moments that matter. Our app skips straight to them: a turning-point finder scans a game's play-by-play data, ranks the plays by how much they swung win probability, and starts you right before each one.

After each call, an AI broadcast booth reacts with two lines of announcer commentary, praising a fourth-down conversion or roasting a bad play.

Functional Requirements

* will be built after the core game works end to end if time allows

1. Scenario selection

As a player,

1. I would like each scenario to open with a brief intro that sets the scene.
2. I would like to start a scenario with one click from a set of random scenarios that updates daily.
3. I would like to have a separate option to play the game endlessly*.
4. I would like to upload a screenshot from a video and start from that exact moment*.

2. Gameplay

As a player,

1. I would like to choose a play call: run, short pass, deep pass, punt, or field goal.
2. I would like each result to be drawn from real plays in similar situations, so outcomes feel realistic.
3. I would like to see the result of each play: yards gained, first down, turnover, or score.
4. I would like the current scenario to continue play by play until it ends in a score, turnover, punt, or the end of the half or game.
5. I would like to see my win probability update after every play.

3. Scoring against the real coach

As a player,

1. I would like to see what the real coach called and what actually happened after each of my decisions
2. I would like a running score comparing my win probability to the real coach's.
3. I would like an end-of-scenario summary with my final grade and best and worst calls.

4. AI broadcast booth

As a player,

1. I would like two lines of announcer commentary after every play reacting to my call and its result.
2. I would like the commentary to reference the real situation (score, clock, field position).
3. I would like the game to keep working if the AI commentary is slow or fails.

5. Screenshot mode*

As a player,

1. I would like the app to read the score bug from my screenshot: teams, score, quarter, clock, and down and distance*.
2. I would like the app to match those details to the exact play in the database to get field position and context*.
3. I would like to confirm or correct the detected situation before the scenario starts*.
4. I would like the commentary to mention what the defense appears to show pre-snap*.

Non-Functional Requirements

1. Performance

As a developer,

1. I would like to play simulation to respond within 300 ms, since it only samples from data already in memory.
2. I would like AI commentary to arrive within 3 seconds, loading separately so it never blocks the next play call.
3. I would like the scenario list to load within 1 second, served from precomputed JSON.
4. I would like screenshot parsing to finish within 10 seconds*.

2. Reliability

As a developer,

1. I would like the game to stay playable if the LLM API fails, falling back to simple template commentary.
2. I would like all play-by-play data and scenarios loaded at startup, so the app never depends on an outside data source during the demo.
3. I would like the simulator to always return a valid result, even for rare situations with few matching plays.

3. Usability

As a developer,

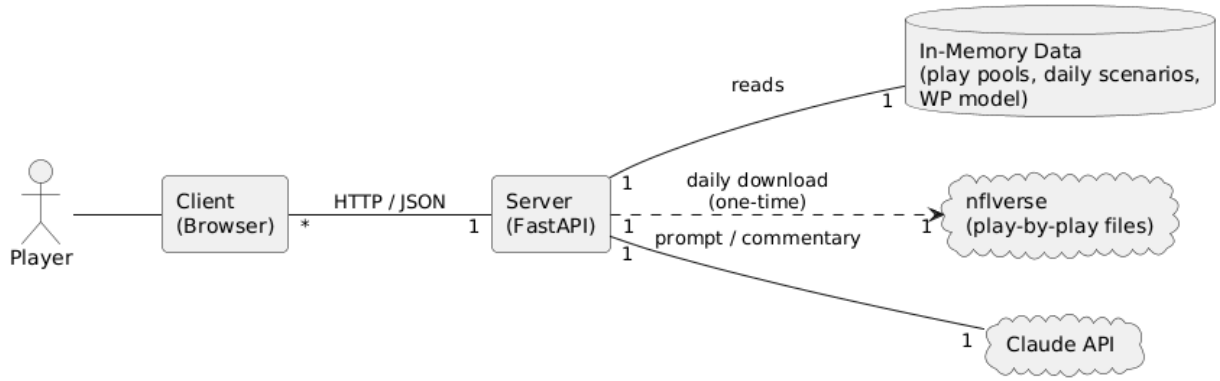
1. I would like a first-time player to be able to make their first call within 15 seconds of opening the app.
2. I would like the game to work on a laptop browser and be usable on a phone.
3. I would like every play to show clear feedback: what happened, the win probability change, and the real coach's call.

4. Security

As a developer,

1. I would like the LLM API key stored only on the server in an environment variable, never sent to the browser.
2. I would like basic rate limiting on AI endpoints so a public demo link can't run up API costs.
3. I would like uploaded screenshots checked for file type and size (under 5 MB), processed in memory, and not stored*.

High level overview of the system



Our implementation will use a client-server architecture with 3 layers: the client-facing browser, the FastAPI server for game logic, and in-memory data layer loaded at startup. Unlike a traditional 3-tier architecture, we do will not use a separate database since the data is read-only and small enough to fit in memory

nflverse contains various datasets relating to NFL, providing the info needed for our game logic and, most importantly, to compute our own win probability (WP) model. While each scenario starts from a real moment in an NFL game, each decision the player makes creates a new scenario with its own down, distance, field position, and win probability, which the server calculates from similar real plays.

Only the server calls the Claude API to keep our key private. To limit calls, each day's scenario intros will be generated once.

Design Issues

Functional Issues

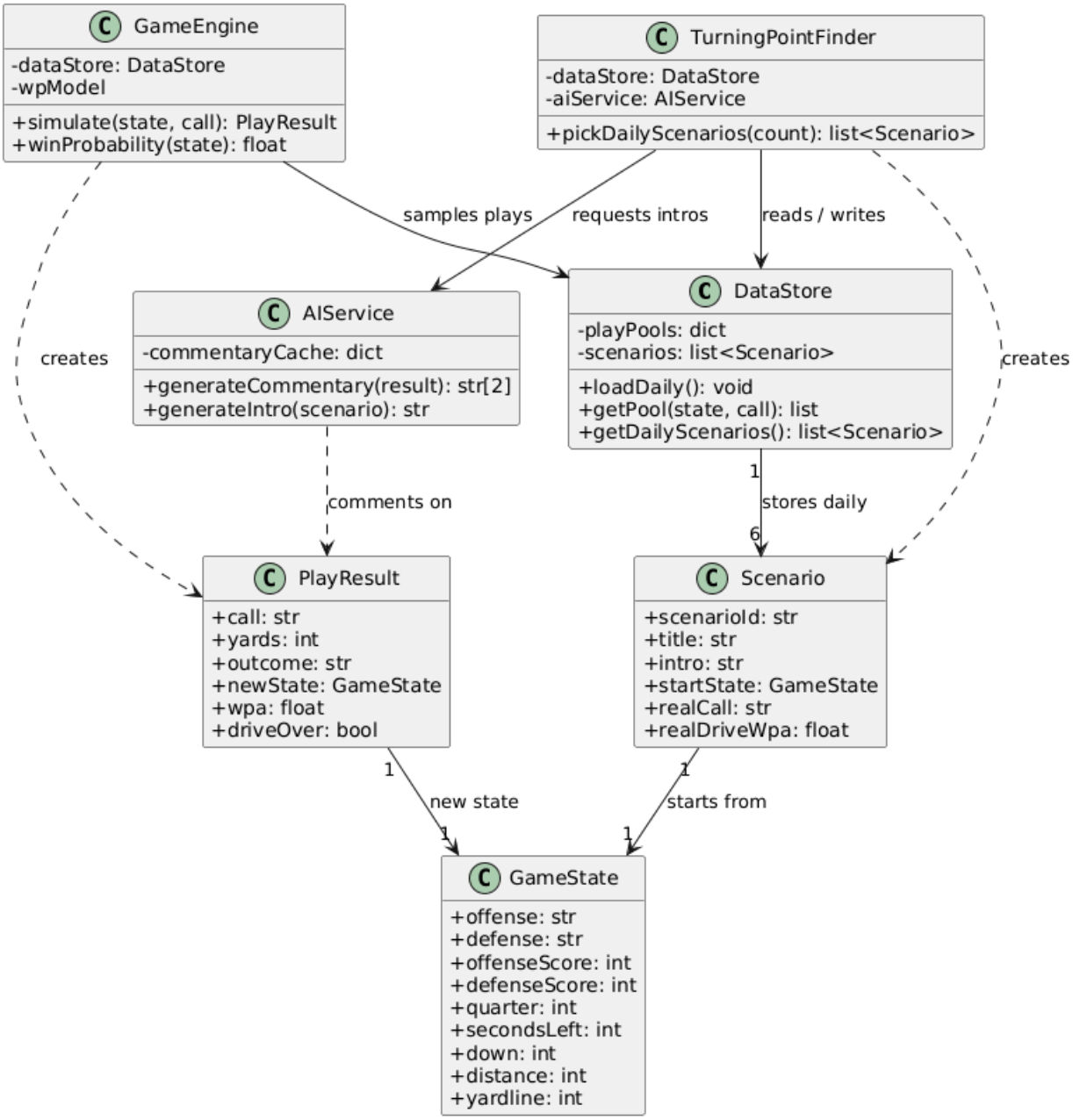
1. Choosing scenarios: we randomly choose the last drive in one possession game. We found that the largest change in WP came from too dramatic of a play e.g. a 90 yard touchdown with 10 seconds to go. We use the LLM to write the intro for each daily scenario.
2. Simulating a play call: we sample a real historical play from similar situations with the same play type to keep outcomes realistic.
3. Scoring against the real coach: we compare the player's total win probability added (WPA) across the scenario with the real coach's. WPA rewards good decisions on an entire drive.
4. Screenshot mode*: A vision model reads only the score bug, then we look up the exact play in our play-by-play data. Vision models read the score bug reliably but often misjudge field position from the yard-line numbers. Teams, quarter, clock, score, and down and distance together nearly always identify a single play, which gives us exact field position and what really happened next.

Non-Functional Issues

1. Backend framework: Python with FastAPI. The data work (nflverse, pandas, scikit-learn) is all in Python, so keeping the backend solely in Python avoids the complexity of using multiple languages and data handoffs. FastAPI is quick to set up and documents its own endpoints.
2. Win probability for new situations: we train our own win probability model on nflverse data. Once a player makes a different call than the real coach, the game reaches states that aren't in the data. A gradient boosting model trained on score difference, time remaining, field position, down, distance, and timeouts covers any state. If it misbehaves, we fall back to average real win probabilities grouped by score, time, and field position.
3. Where data lives: loaded to memory from files downloaded once per day. The data is read-only and only the useful columns are added so it is small enough to fit in memory. Skipping a database saves setup time, keeps every play fast, and removes a point of failure.
4. AI model: the Claude API. We need fast, reliable text generation with structured JSON output. Generating intros once per day while caching commentary keeps the cost low.

Design Details

Descriptions of Classes and Interaction between Classes



- DataStore
    - Holds the day's data in memory: the play pools grouped by situation and the 6 daily Scenarios.
    - Is loaded once per day through loadDaily(), which reads the downloaded nflverse data and builds the play pools.

    - Gives GameEngine the matching pool of real plays for a game state and play call through getPool().
    - Returns the 6 daily Scenarios to the client through getDailyScenarios().
- GameState
    - Represents the situation before a play: offense, defense, both scores, quarter, seconds left, down, distance, and yard line.
    - Is the starting point of every Scenario and is returned as the new state inside every PlayResult.
    - Is passed into GameEngine to simulate a play and to compute win probability.
- Scenario
    - Represents one playable moment from a real NFL game.
    - Contains a unique scenario ID, a title, an AI-written intro, the starting GameState, the real coach's call, and the real coach's total WPA on that drive.
    - Is created by TurningPointFinder and stored in DataStore, which holds 6 per day.
    - Uses realCall and realDriveWpa to compare the player's decisions with the real coach's.
- PlayResult
    - Represents the outcome of one play call: the call, yards gained, outcome type, the new GameState, the WPA from the play, and whether the drive is over.
    - Is created by GameEngine after every play and returned to the client.
    - Is passed to AIService to generate commentary.
- GameEngine
    - Runs the game logic for each play and holds the trained win probability model.
    - simulate() takes a GameState and a play call, gets the matching pool from DataStore, samples a real play, applies it, and returns a PlayResult.
    - winProbability() returns the offense's chance of winning from a GameState, used before and after each play to compute WPA.
- TurningPointFinder
    - Picks the 6 daily Scenarios through pickDailyScenarios().
    - Reads plays from DataStore, filters out garbage time, ranks plays by WPA and leverage, and keeps the top play per drive.
    - Asks AIService to write an intro for each Scenario, then stores the Scenarios in DataStore.
- AIService
    - The only class that calls the Claude API.
    - generateIntro() writes a short scene-setting intro for each daily Scenario, once per day.
    - generateCommentary() takes a PlayResult and returns two announcer lines.
    - Caches commentary for repeated situations and uses template lines if the API is slow or fails.

Data source

nflverse play-by-play for the 2024 and 2025 seasons, specifically using columns: `game_id`, `play_id`, `season`, `week`, `home_team`, `away_team`, `posteam`, `defteam`, `qtr`, `game_seconds_remaining`, `half_seconds_remaining`, `down`, `ydstogo`, `yardline_100`, `score_differential`, `posteam_timeouts_remaining`, `defteam_timeouts_remaining`, `play_type`, `pass_length`, `yards_gained`, `touchdown`, `interception`, `fumble_lost`, `sack`, `field_goal_result`, `kick_distance`, `wp`, `wpa`, `desc`, `drive`, `total_home_score`, `total_away_score`, `result`.

Data Models

Model	Fields	Purpose
`GameState`	offense, defense, offense_score, defense_score, quarter, seconds_left_in_game, down, distance, yardline_100, offense_timeouts, defense_timeouts	The situation before a play. `yardline_100` = yards from the opponent's end zone, matching nflverse.
`PlayCall`	enum: `run`, `short_pass`, `deep_pass`, `punt`, `field_goal`	The player's decision. Going for it on 4th = calling a run or pass on 4th down.
`PlayResult`	call, yards, outcome (`gain`, `first_down`, `touchdown`, `turnover`, `turnover_on_downs`, `punt`, `fg_made`, `fg_missed`), description, sampled_play_id	What happened, plus the real play it was sampled from.
`SimulateResponse`	result, new_state, wp_before, wp_after, wpa, drive_over	Returned after every call.
`Scenario`	id, game_id, title, intro, start_state, real_call, real_play_desc, real_drive_wpa, leverage_score	One playable starting moment. `real_drive_wpa` = the real coach's total WPA from this play to the end of the drive.
`Commentary`	line_1, line_2	Two announcer lines.

Play simulator

Given a `GameState` and a `PlayCall`, sample a real outcome.

Situation buckets:

- Down: 1, 2, 3, 4
- Distance: 1, 2 to 3, 4 to 6, 7 to 10, 11+
- Field zone by `yardline_100`: 80 to 99 (backed up), 50 to 79 (own side), 21 to 49 (opponent side), 1 to 20 (red zone)

By call:

- `run`: sample from `play_type == run` in the same bucket.
- `short_pass` and `deep_pass`: sample from `play_type == pass` with matching `pass_length`. Sacks have no `pass_length`, so add sacks from the bucket to both pools at their real rate.
- `punt`: sample net yards from real punts in the same field zone.
- `field_goal`: kick distance = `yardline_100 + 17`. Make chance = real make rate for kicks within 3 yards of that distance.

Fallbacks: if a bucket has fewer than 30 plays, drop the field zone, then merge distance buckets, until there are at least 30.

Applying a result: cap yards at the end zone (touchdown) and at a safety. Update down and distance, flip possession on turnovers, punts, and turnover on downs. To simulate the clock: subtract 30 seconds for a run or completion, 6 for an incompletion, and 15 for any play in the last 2 minutes of a half.

Drive ends on a touchdown, field goal attempt, turnover, punt, turnover on downs, or the clock running out in the half.

WP model

- Features: score differential, game seconds remaining, score differential divided by (seconds remaining + 1), yardline_100, down, distance, offense timeouts, defense timeouts.
- Label: whether the team with the ball won (from `result` and the home/away teams). Ties count as 0.5 weight on each class, or drop them.
- Model: scikit-learn `HistGradientBoostingClassifier` (handles the nonlinear late-game effects better than logistic regression and trains in about a minute). Save with joblib.
- Sanity check: compare our WP with nflverse's `wp` column on held-out games. Within about 0.05 on average is good enough.
- Fallback: average nflverse `wp` by score differential, time bucket, and field zone.
- For a state where the defense now has the ball, WP for the player's team = 1 minus the model's output for the new offense.

API endpoints

Method	Path	Body or params	Returns
GET	`/api/scenarios`	none	Curated `Scenario` list from `scenarios.json`
GET	`/api/games`	`season`, `team` (optional)	Games with teams, week, final score
GET	`/api/games/{game_id}/turning-points`	`limit` (default 5)	`Scenario` list for that game
POST	`/api/simulate`	`state`, `call`	`SimulateResponse`
POST	`/api/commentary`	`state_before`, `call`, `result`, `wp_before`, `wp_after`	`Commentary`
POST	`/api/scenario-from-screenshot` (stretch)	image file	Detected fields plus the matched `Scenario`

Prompts for Claude API

1. Commentary

System prompt: You are a two-person NFL TV broadcast booth: line_1 is the play-by-play announcer, line_2 is the color analyst. React to the coach's call and its result, referencing the real situation (score, clock, field position). Hype good aggressive calls, roast conservative calls that backfire, stay PG. Each line must be under 25 words. Reply only with the JSON object.

User prompt template:
Situation: {quarter} {clock}, {score}, {down & distance} at {field position}, timeouts {off}-{def}.
Call: {run | short pass | deep pass | punt | field goal}.
Result: {sampled real play description} (outcome: {outcome}).
Win probability for {offense}: {wp_before}% -> {wp_after}%.

Example:
Situation: Q3 10:00, CRM trails NVY 17-20, 2nd & 7 at the NVY 40, timeouts 3-3.
Call: short pass.
Result: J.Smith pass short right to T.Jones for 9 yards (outcome: first_down).
Win probability for CRM: 41% -> 47%.

Output format: a JSON schema forces the reply to be {"line_1": string, "line_2": string}.

2. Scenario Introduction

System prompt: You are an NFL TV play-by-play announcer setting the scene right before a snap. Write 2 to 3 sentences in broadcast voice using the situation given. End right before the snap. Never reveal or hint at what happened on the play. Reply only with the JSON object.

User prompt template:
Situation: {same situation line as above}.
What really happened (DO NOT reveal or hint at this): {real play description}

Output format: {"intro": string}

Repo Structure Outline

```
None
Scrimmagine/
├── backend/               # FastAPI server: game logic, win
probability model, Claude API calls
├── frontend/              # HTML/CSS/JS game UI
├── sprites/               # pixel-art player and ball sprites
├── sound-effects/         # music and sound effects
├── scrimmagine-mockups/   # UI mockups from Claude Design
└── Scrimmagine Design Doc.pdf
```

Rules

- Load all data once at startup, never per request.
- Never expose the API key to the client.
- Every AI call needs a timeout and a non-AI fallback, so the game still works offline.

UI Design

We wanted the theme to be retro, so we took inspiration from games like Krillion and Eraly

Claude Design was used to create UI mockups

Primary and secondary colors of each team:

- Buffalo Bills (BUF): Primary #00338D, Secondary #C60C30
- Miami Dolphins (MIA): Primary #008E97, Secondary #FC4C02
- New England Patriots (NE): Primary #002244, Secondary #C60C30
- New York Jets (NYJ): Primary #125740, Secondary #000000
- Baltimore Ravens (BAL): Primary #241773, Secondary #000000
- Cincinnati Bengals (CIN): Primary #FB4F14, Secondary #000000
- Cleveland Browns (CLE): Primary #311D00, Secondary #FF3C00
- Pittsburgh Steelers (PIT): Primary #FFB612, Secondary #101820
- Houston Texans (HOU): Primary #03202F, Secondary #A71930
- Indianapolis Colts (IND): Primary #002C5F, Secondary #FFFFFF
- Jacksonville Jaguars (JAX): Primary #006778, Secondary #101820
- Tennessee Titans (TEN): Primary #0C2340, Secondary #4B92DB
- Denver Broncos (DEN): Primary #FB4F14, Secondary #002244
- Kansas City Chiefs (KC): Primary #E31837, Secondary #FFB81C
- Las Vegas Raiders (LV): Primary #A5ACAF, Secondary #000000
- Los Angeles Chargers (LAC): Primary #0080C6, Secondary #FFC20E
- Dallas Cowboys (DAL): Primary #003594, Secondary #869397
- New York Giants (NYG): Primary #0B2265, Secondary #A71930
- Philadelphia Eagles (PHI): Primary #004C54, Secondary #A5ACAF
- Washington Commanders (WAS): Primary #5A1414, Secondary #FFB612
- Chicago Bears (CHI): Primary #0B162A, Secondary #C83803
- Detroit Lions (DET): Primary #0076B6, Secondary #B0B7BC
- Green Bay Packers (GB): Primary #203731, Secondary #FFB612
- Minnesota Vikings (MIN): Primary #4F2683, Secondary #FFC62F
- Atlanta Falcons (ATL): Primary #A71930, Secondary #000000
- Carolina Panthers (CAR): Primary #0085CA, Secondary #101820
- New Orleans Saints (NO): Primary #D3BC8D, Secondary #101820
- Tampa Bay Buccaneers (TB): Primary #D50A0A, Secondary #34302B
- Arizona Cardinals (ARI): Primary #97233F, Secondary #000000
- Los Angeles Rams (LA): Primary #003594, Secondary #FFA300
- San Francisco 49ers (SF): Primary #AA0000, Secondary #B3995D
- Seattle Seahawks (SEA): Primary #002244, Secondary #69BE28

Sound Design

- “main_menu.mp3” plays only in the menu and loops indefinitely
- “first_down.mp3” plays only when a first down is reached
- “failed_play.mp3” plays only when a play fails
- “button.mp3” plays when any button is pressed
- “game_over.mp3” plays when grade is D or below at the end of screen of scenario
- “win.mp3” plays when grade is B or above at the end of screen of scenario

Testing/Auditing

- Automatic unit testing via pytest
- Refer to test log to see system testing
- Load testing if time allows*

In the future, use OpenAI to audit public repo or zip file for private repo*

Deployment

We are planning on deploying to HuggingFace if time allows.

Backend prompt for Claude Code:

You are a senior Python engineer building the backend for a hackathon game in an 8-hour window. Favor simple, typed, explainable code over clever code. We will walk judges through it.

<context>
Repo root: Indy_Init/. Current contents:
- "Indy Init Design Doc.pdf": the full spec and the source of truth for data models, endpoints, simulator rules, the WP model, the turning point finder, AI prompts, repo structure and build order. Read ALL of it before writing any code.
- scrimmagine-mockups/: static HTML UI mockups made in Claude Design. You are NOT building the UI. Read them only to confirm the API returns everything the screens need.
- sprites/: pixel-art player sprites used by the frontend.
Game: "Scrimmagine". Players make play calls at turning points of real NFL games and get scored against the real coach using win probability added (WPA).
Workflow: the backend is built here in Claude Code. The UI was designed separately in Claude Design and will be built in a later session against this API.
</context>

<task>
Create backend/ exactly as specified in the doc's "Repo structure" section, following its "Build order" steps 1 to 7. Skip step 8 (screenshot endpoint) unless I ask for it later.
</task>

<decisions_already_made>
- Python + FastAPI + Pydantic + pandas + scikit-learn (HistGradientBoostingClassifier) + joblib + nflreadpy + the anthropic SDK. No database. All data is loaded into memory once at startup.
- Use snake_case field names from the doc's Pydantic table, not the camelCase names in the UML diagram.
- The doc says 5 turning points in one place and 6 daily scenarios in another. Use 6 for the daily set from GET /api/scenarios and limit=5 as the default for /api/games/{game_id}/turning-points.
- Read the API key from ANTHROPIC_API_KEY and the model name from CLAUDE_MODEL (put both in backend/.env.example). Never hardcode either.
- Allow CORS for http://localhost:8000 and http://127.0.0.1:5500 (VS Code Live Server).
- Mount the repo-root sprites/ folder at /sprites and the repo-root frontend/ folder at / (only if it exists), so one server runs the whole game later.
</decisions_already_made>

<constraints>
- MUST load the parquet file, the WP model and scenarios.json once at startup, never per request.
- MUST give every Claude API call a 3 second timeout and a template fallback, so the game works fully offline.

- MUST make /api/simulate always return a valid result, using the bucket fallback rules from the doc.
- MUST keep the API contract exactly as the doc's endpoint table and models describe.
- Only build what the doc specifies. Do NOT add auth, a database, logging frameworks, Docker files or extra endpoints.
- Keep functions small with type hints and a one-line docstring each.
- NEVER edit or move "Indy Init Design Doc.pdf", scrimmagine-mockups/ or sprites/.
</constraints>

<stop_and_ask_before>
- Adding any dependency not listed above
- Creating or changing any file outside backend/ (except adding backend/data/ to .gitignore)
- Changing any endpoint path, request shape or response shape from the doc
- Any step that downloads more than the 2024 and 2025 play-by-play seasons
</stop_and_ask_before>

<process>
Work through the build order in sequence without waiting for me between steps. After each step, print one line: "✅ Step N: <what works, how you verified it>".
If the same error survives 2 fix attempts, stop and report the error, what you tried and your best hypothesis.
</process>

<done_when>
- pytest passes, covering: every play call returns a valid SimulateResponse; buckets fall back correctly when a bucket has fewer than 30 plays; touchdowns, turnovers, punts and turnovers on downs flip possession correctly; the turning point finder skips plays with WP below 0.05 or above 0.95 and keeps at most one play per drive.
- The WP sanity check prints a mean absolute difference from nflverse wp on held-out games (target: about 0.05 or less).
- `uvicorn app.main:app` (run from backend/) starts, and /sprites/crimson_ready.png loads in the browser.
- curl against GET /api/scenarios, POST /api/simulate and POST /api/commentary returns JSON matching the doc's models. /api/commentary still returns template lines when ANTHROPIC_API_KEY is unset.
- You finish with backend/README.md: setup commands, the order to run the scripts, and the curl examples you used.
</done_when>

Frontend prompt for Claude Code:

You are a front-end engineer turning finished Claude Design mockups into the working UI for "Scrimmagine", a retro pixel-art NFL play-calling game. The FastAPI backend in backend/ is done and its API contract is frozen.

<context>
Repo root: Indy_Init/.
- Spec: "Indy Init Design Doc.pdf". Read the Purpose, Functional Requirements, "UI Design" and "Sequence of events" sections, plus the API endpoints table and data models.
- Visual source of truth: scrimmagine-mockups/*.html. Read every file before writing code. They are static exports, so copy their markup and inline styles closely: colors, fonts (Press Start 2P + VT323), 4px black pixel borders, hard drop shadows, image-rendering: pixelated.
- Screen map: Main.html = home + daily scenarios, Gameplay.html = pre-snap call, PlayResult.html = converted play, PlayFail.html = failed 4th down / turnover, Summary.html = drive over, Mobile.html = phone layout of Gameplay. Screenshot.html is a stretch goal. Do not build it yet.
- Sprites: sprites/{crimson,navy,forest,midnight}_{ready,stance,run,throw,celebrate}.png and sprites/ball.png (20x24 px). The backend serves them at /sprites/. Reference them as /sprites/<file>, not the mockups' relative paths. Render at 2x on desktop and 1.5x on phone. The offense faces right; mirror the defense with scaleX(-1).
- The mockups use made-up teams (CRM, NVY...) and numbers. Replace all of them with live API data.
</context>

<task>
Build frontend/ at the repo root as plain HTML, CSS and vanilla JS ES modules (no framework, no build step). The backend already serves frontend/ at /. Use one index.html with screen sections toggled by JS state, since the screens share game state.
</task>

<behavior>
- Home: GET /api/scenarios, then render the 6 cards. Clicking a card starts that scenario.
- Gameplay: show the score bug, a top-down field drawn from yardline_100 (blue line of scrimmage, yellow line to gain), the formation sprites, the scenario intro, a WP bar and the call buttons. Enable only legal calls: punt and field goal only on 4th down, unless under 2:00 in the 2nd or 4th quarter.
- On a call: POST /api/simulate, then play a roughly 1.2s sprite animation of the result (throw/run frames, ball path, run toward the new spot), then show the result banner, WP before → after and the WPA chip. Fire POST /api/commentary at the same time and show the booth lines when they arrive, never blocking the next call. Use the PlayResult.html style for a conversion and the PlayFail.html style for failures.

- When drive_over is true, show Summary: your total WPA vs real_drive_wpa, the real call and real_play_desc, a per-play log with the best and worst calls tagged, and a grade from (your WPA minus coach WPA, in points): ≥10 A, ≥3 B, ≥−3 C, ≥−10 D, else F.
- Team colors: map each NFL team abbreviation to the closest of the 4 sprite colorways in a single frontend/teamColors.js config file.
- The layout MUST work at 1280px and at 390px (Mobile.html is the phone reference). Make every button a real <button> at least 44px tall.
</behavior>

<constraints>
- Only create or edit files inside frontend/. NEVER edit backend/, sprites/, scrimmagine-mockups/ or the design doc.
- Do NOT add npm, frameworks, bundlers, analytics or features not listed.
- Stop and ask before changing any API call shape or touching any file outside frontend/.
- If the same bug survives 2 fix attempts, stop and report.
</constraints>

<done_when>
- With `uvicorn app.main:app` running, http://localhost:8000 loads the game, and a first-time player can pick a scenario and make a call within 15 seconds.
- A full drive plays end to end through Summary for at least 2 different scenarios, with no console errors.
- The game stays playable with ANTHROPIC_API_KEY unset (template commentary appears).
- After each major screen works, print "✅ <screen>: <how you verified it>". Finish with a list of any mockup details you could not match and why.